```
mov al, 200
mov bl, -1
cmp al, bl      ⇐ compares nothing
```

jl et1 ; -56 < -1

→ Signed interp.

⇓

The jump happens.

a < b

⇓

a - b < 0

200 < -1

ja et2 200 > 255?

→ unsigned interp.

⇓

it will **not** happen.

jb et3 200 < 255

⇓

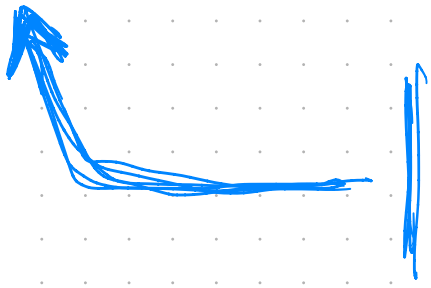
it will happen.

jg et4 -56 > -1

⇓ it will not happen.

⇓

YOU CANNOT COMPARE 200 W/ −1 ON 1 BYTE.



(−128, 127) ← −1 is here

(0, 256) ← 200 is here.

(−32768, 32767)

↑ ↑ ↑ ↑ ↘

YOU MUST KNOW THOSE signed word

⇒ mov cx, 200

mov bx, −1

cmp cx, bx WORKS (200 > −1).

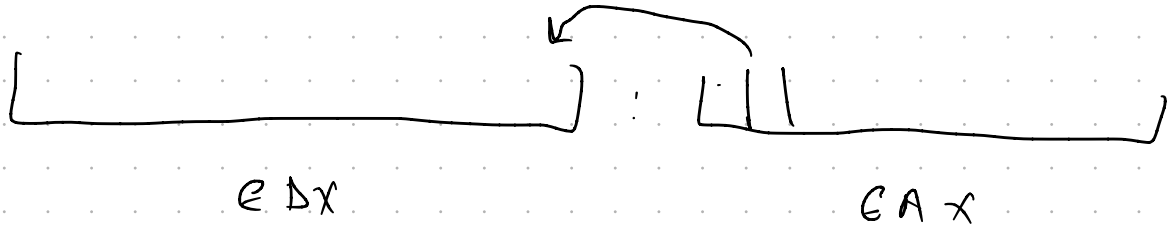
xor edx, edx

mov dl, 0fh

? write a following seg which multiplies by 4 the value EDX:EAX

↓

"shl EDX:EAX, 2"



```
shl eax,1
rcl edx,1      put the CF there.
shl eax,1
rcl edx,1
```

location counter and pointer arithmetic.

```
0  a   db  1,2,3,4 ; 01 02 03 04
4  lg  db  $-a     ; 04
5      db  a-$     ; -5 = FB
6  c   equ c-$     ; -6 = FA      (OR)
   e   db  a-$
   x   dw  x                    07 00   in olly dbg, POV 07 20
```

in ASSEMBLER POV.

```
x1  db  x1   ← syntax error. (like dword/word on stack)

db  lg-a     04
db  a-lg     -4 = FC
db  [$-a]    ← syntax error.
```

db [lg - a]

You cannot use initializing values in assembly time.

Ind. of expression = CONSTANT DETERMINABLE AT ASSEMBLY TIME.

this is not runtime. Same with db [eax]. WRONG.

lg1 EQU lg1 ← syntax in some cases recursive.

↓

NASM BUG !!! => lg1 = 0.

lg1 EQU lg1 - a => lg1 = 0. a=0 offset.

NORMALLY SYNTAX ERROR,

BUT NASM LETS IT GO.

lg1 EQU lg1 - e => syntax error.

b dd a - start ; syntax error.

(here - somewhere else)

dd start - a (somewhere - here) ✓

dd start - start1 ✓ ✓

mov ah, lg1 ; = 0

mov ah, lg ; syntax error! (offset into a byte)

mov ch, lg.a ; ✓

mov ch, [lg-a]

mov cx, $-a ⟹ syntax error.

here. somewhere else.

mov ch, a-$ - somewhere else - here ✓✓

NOT BYTE!

contact determinable at comp. time ⇒ only offsets from the current segments.

mov cx, a-$ ✓✓

← label in the code segment

mov cx, $-start
mov cx, start-$ | good

mov ch, $-start.
mov ch, start-$ } ⇒ scalar. (they are from the same segment)

mov ecx, a - start. ✓

start - a. ✗

mov ah, a + b (no syntax error)

a + b = (a - $$) + (b - $$)

mov ex, [a+b] ← syntax error.

SOMEWHERE ELSE - HERE OK

OTHERWISE NOT.